1. Methods

practice.ts

```
export {};
class Greeter {
greet(name: string) {
console.log(`${name}, do your stuff!`);
}
}
new Greeter().greet("Miss Frizzle"); // This will run successfully!
// new Greeter().greet();
// ^ If you remove the // from the line above, TypeScript will throw an error:
// "Expected 1 arguments, but got 0."
// --- Class Constructors Example ---
class Greeted {
constructor(message: string) {
console.log(`As I always say: ${message}!`);
}
}
new Greeted("take chances, make mistakes, get messy"); // This will run successfully!
// new Greeted();
// ^ If you remove the // from the line above, TypeScript will throw an error:
// "Expected 1 arguments, but got 0."
```

TERMINAL

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Miss Frizzle, do your stuff!
As I always say: take chances, make mistakes, get messy!
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

2. Properties

practice.ts

```
export {}; // Keeps our file isolated!
class FieldTrip {
// 1. The Blueprint: We explicitly declare the property here first
destination: string;
constructor(destination: string) {
// 2. Now we are allowed to assign a value to it!
this.destination = destination;
console.log(`We're going to ${this.destination}!`);
// We completely removed the `this.nonexistent` line because
// it breaks TypeScript's strict blueprint rules!
}
}
// 3. Execution: Let's actually build the trip
const trip = new FieldTrip("the planetarium");
// 4. We can safely read the property from the outside, too
console.log(`Our trip destination is saved as: ${trip.destination}`);
```

TERMINAL

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Miss Frizzle, do your stuff!
As I always say: take chances, make mistakes, get messy!
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
We're going to the planetarium!
Our trip destination is saved as: the planetarium
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

3. Function Properties

```typescript
class WithProperty {
}
const arrowInstance1 = new WithProperty();
const arrowInstance2 = new WithProperty();
console.log(
"Are arrow properties shared in memory?",
arrowInstance1.myProperty === arrowInstance2.myProperty
); // This will print: false
// --- 3. Function Property with Parameters ---
class WithPropertyParameters {
// We strictly enforce that 'input' MUST be a boolean (true/false)
takesParameters = (input: boolean) => input ? "Yes" : "No";
}
const testInstance = new WithPropertyParameters();
console.log("Did the boolean parameter work?", testInstance.takesParameters(true));
// testInstance.takesParameters(123);
// ^ If you remove the //, TypeScript will yell:
// "Argument of type 'number' is not assignable to parameter of type 'boolean'."n
```

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Miss Frizzle, do your stuff!
As I always say: take chances, make mistakes, get messy!
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
We're going to the planetarium!
Our trip destination is saved as: the planetarium
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Are standard methods shared in memory? true
Are arrow properties shared in memory? false
Did the boolean parameter work? Yes
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

4. Initialization checking

```typescript
export {}; // Keeps our file isolated!

// --- Initialization Checking ---

class WithValue {
    immediate = 0; // Ok - assigned directly
    later: number; // Ok - set in the constructor
    mayBeUndefined: number | undefined; // Ok - allowed to be undefined

    // unused: number;
    // ^ If you remove the //, TypeScript will throw an error:
    // "Property 'unused' has no initializer
    // and is not definitely assigned in the constructor."

    constructor() {
        this.later = 1;
    }
}

const example = new WithValue();
console.log("immediate:", example.immediate);
console.log("later:", example.later);
console.log("mayBeUndefined:", example.mayBeUndefined);

// --- MissingInitializer Example ---
// The code below is commented out because it would crash at runtime!

// class MissingInitializer {
//     property: string;
// }
// new MissingInitializer().property.length;
// ^ TypeError: Cannot read property 'length' of undefined
// This is what strict initialization checking protects us from!

// --- Definitely Assigned Properties ---
```

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
immediate: 0
later: 1
mayBeUndefined: undefined
First activity popped: learn
Second activity popped: sleep
Remaining activities: [ 'eat' ]
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

5. Optional properties

practice.ts

```typescript
export {}; // Keeps our file isolated!

// --- Optional Properties ---

class MissingInitializer {
    property?: string;
}

new MissingInitializer().property?.length; // Ok

// new MissingInitializer().property.length;
// ^ If you remove the //, TypeScript will throw an error:
// "Object is possibly 'undefined'."
```

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
immediate: 0
later: 1
mayBeUndefined: undefined
First activity popped: learn
Second activity popped: sleep
Remaining activities: [ 'eat' ]
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx tsc practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE> node practice.js
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

6. Read only properties

practice.ts

```typescript
export {};

// --- Read-Only Properties ---

class Quote {
    readonly text: string;

    constructor(text: string) {
        this.text = text;
    }

    emphasize() {
        // this.text += "!";
        // Error: Cannot assign to 'text' because it is a read-only property.
    }
}

const quote = new Quote(
    "There is a brilliant child locked inside every student."
);

// quote.text = "Ha!";
// Error: Cannot assign to 'text' because it is a read-only property.

class RandomQuote {
    readonly explicit: string = "Home is the nicest word there is.";
    readonly implicit = "Home is the nicest word there is.";

    constructor() {
        if (Math.random() > 0.5) {
            this.explicit = "We start learning the minute we're born."; // Ok

            // this.implicit = "We start learning the minute we're born.";
            // Error: Type '"We start learning the minute we're born."' is
            // not assignable to type '"Home is the nicest word there is."'.
        }
    }
}

const randomQuote = new RandomQuote();

randomQuote.explicit; // Type: string
randomQuote.implicit; // Type: "Home is the nicest word there is."
```

```typescript
export {};

// --- Read-Only Properties ---

class Quote {
    readonly text: string;

    constructor(text: string) {
        this.text = text;
    }

    emphasize() {
        // this.text += "!";
        // Error: Cannot assign to 'text' because it is a read-only property.
    }
}

const quote = new Quote(
    "There is a brilliant child locked inside every student."
);

// quote.text = "Ha!";
// Error: Cannot assign to 'text' because it is a read-only property.

class RandomQuote {
```

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

7. Classes as types

```typescript
export {};

class Teacher {
    sayHello() {
        console.log("Take chances, make mistakes, get messy!");
    }
}

let teacher: Teacher;

teacher = new Teacher(); // Ok
teacher.sayHello();

// teacher = "Wahoo!";
// Error: Type 'string' is not assignable to type 'Teacher'.

class SchoolBus {
    getAbilities() {
        return ["magic", "shapeshifting"];
    }
}

function withSchoolBus(bus: SchoolBus) {
    console.log(bus.getAbilities());
}

withSchoolBus(new SchoolBus()); // Ok

// Ok
withSchoolBus({
    getAbilities: () => ["transmogrification"],
});

// withSchoolBus({
//     getAbilities: () => 123,
// });
// Error: Type 'number' is not assignable to type 'string[]'.
```

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Take chances, make mistakes, get messy!
[ 'magic', 'shapeshifting' ]
[ 'transmogrification' ]
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

8. Classes and interfaces

EXPLORER

TS-PRACTICE

JS practice.js

TS practice.ts

TS practice.ts

TS practice.ts > ...

```ts
export {};

interface Learner {
    name: string;
    study(hours: number): void;
}

class Student implements Learner {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    study(hours: number) {
        for (let i = 0; i < hours; i += 1) {
            console.log("...studying...");
        }
    }
}

// class Slacker implements Learner {
//     // Error: Class 'Slacker' incorrectly implements interface 'Learner'.
//     //  Property 'study' is missing in type 'Slacker'
//     //  but required in type 'Learner'.
//     name = "Rocky";
// }

const student = new Student("Ms. Frizzle");
console.log(student.name);
student.study(3);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Take chances, make mistakes, get messy!
[ 'magic', 'shapeshifting' ]
[ 'transmogrification' ]
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Ms. Frizzle
...studying...
...studying...
...studying...
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

OUTLINE

TIMELINE

9. Implementing multiple interfaces

EXPLORER

TS-PRACTICE

- practice.js
- practice.ts

TS practice.ts

TS practice.ts > ...

```ts
export {};

interface Graded {
    grades: number[];
}

interface Reporter {
    report: () => string;
}

class ReportCard implements Graded, Reporter {
    grades: number[];

    constructor(grades: number[]) {
        this.grades = grades;
    }

    report() {
        return this.grades.join(", ");
    }
}

// class Empty implements Graded, Reporter { }
// Error: Class 'Empty' incorrectly implements interface 'Graded'.
//   Property 'grades' is missing in type 'Empty'
//   but required in type 'Graded'.
// Error: Class 'Empty' incorrectly implements interface 'Reporter'.
//   Property 'report' is missing in type 'Empty'
//   but required in type 'Reporter'.

const card = new ReportCard([90, 85, 100]);
console.log(card.report());

// --- Conflicting Interfaces ---
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Take chances, make mistakes, get messy!
[ 'magic', 'shapeshifting' ]
[ 'transmogrification' ]
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Ms. Frizzle
...studying...
...studying...
...studying...
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
90, 85, 100
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

OUTLINE

TIMELINE

10. Extending a class

EXPLORER

TS-PRACTICE

- practice.js
- practice.ts

TS practice.ts

TS practice.ts > OnlineLesson > constructor

```typescript
class OnlineLesson extends Lesson {

    constructor(subject: string, url: string) {
        super(subject);
        this.url = url;
    }
}

let lesson: Lesson;
lesson = new Lesson("coding"); // Ok
lesson = new OnlineLesson("coding", "oreilly.com"); // Ok

let online: OnlineLesson;
online = new OnlineLesson("coding", "oreilly.com"); // Ok

// online = new Lesson("coding");
// Error: Property 'url' is missing in type
// 'Lesson' but required in type 'OnlineLesson'.

console.log(lesson.subject);
console.log(online.url);

// --- Structural Subclass Assignability ---

class PastGrades {
    grades: number[] = [];
}

class LabeledPastGrades extends PastGrades {
    label?: string;
}

let subClass: LabeledPastGrades;

subClass = new LabeledPastGrades(); // Ok
subClass = new PastGrades(); // Ok
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
90, 85, 100
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
The surest test of discipline is its absence.
I cannot afford the luxury of a closed mind.
coding
oreilly.com
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

OUTLINE

TIMELINE

EXPLORER

TS-PRACTICE

- practice.js
- practice.ts

TS practice.ts

TS practice.ts > ...

```typescript
export {};

// --- Extending a Class ---

class Teacher {
    teach() {
        console.log("The surest test of discipline is its absence.");
    }
}

class StudentTeacher extends Teacher {
    learn() {
        console.log("I cannot afford the luxury of a closed mind.");
    }
}

const teacher = new StudentTeacher();
teacher.teach(); // Ok (defined on base)
teacher.learn(); // Ok (defined on subclass)

// teacher.other();
// Error: Property 'other' does not exist on type 'StudentTeacher'.

// --- Extension Assignability ---

class Lesson {
    subject: string;

    constructor(subject: string) {
        this.subject = subject;
    }
}

class OnlineLesson extends Lesson {
    url: string;

    constructor(subject: string, url: string) {
        super(subject);
        this.url = url;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
90, 85, 100
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
The surest test of discipline is its absence.
I cannot afford the luxury of a closed mind.
coding
oreilly.com
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

OUTLINE

TIMELINE

11. Overridden constructors

```
TS practice.ts ×
TS practice.ts > ...
29  class GradesTally {
36  }
37
38  class ContinuedGradesTally extends GradesTally {
39      constructor(previousGrades: number[]) {
40          super();
41          // this.grades must come AFTER super()
42          this.grades = [...previousGrades];
43          console.log("Starting with length", this.grades.length); // Ok
44      }
45  }
46
47  const tally = new ContinuedGradesTally([90, 85, 100]);
48  console.log(tally.grades);
49
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\JCool\AppDev\TS-PRACTICE>


Ms. Frizzle
...studying... ...
The surest test of discipline is its absence.
I cannot afford the luxury of a closed mind.
coding
oreilly.com
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
You pass!
Starting with length 3
[ 90, 85, 100 ]
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

12. Overridden properties

```ts
export {};

// --- Overridden Properties ---

class Assignment {
    grade?: number;
}

class GradedAssignment extends Assignment {
    grade: number;

    constructor(grade: number) {
        super();
        this.grade = grade;
    }
}

class NumericGrade {
    value = 0;
}

// class VagueGrade extends NumericGrade {
//     value = Math.random() > 0.5 ? 1 : "...";
// }
// Error: Property 'value' in type 'VagueGrade' is not
// assignable to the same property in base type 'NumericGrade'.
//   Type 'string | number' is not assignable to type 'number'.
//     Type 'string' is not assignable to type 'number'.

// const instance: NumericGrade = new VagueGrade();
// Expected type: number
// Actual type: number | string
// instance.value;
```

```
I cannot afford the luxury of a closed mind.
coding
oreilly.com
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
You pass!
Starting with length 3
[ 90, 85, 100 ]
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

13. Abstract classes

```ts
export {};

// --- Abstract Classes ---

abstract class School {
    readonly name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract getStudentTypes(): string[];
}

class Preschool extends School {
    getStudentTypes() {
        return ["preschooler"];
    }
}

// class Absence extends School { }
// Error: Nonabstract class 'Absence' does not implement
// inherited abstract member 'getStudentTypes' from class 'School'.

let school: School;

school = new Preschool("Sunnyside Daycare"); // Ok
console.log(school.name);
console.log(school.getStudentTypes());

// school = new School("somewhere else");
// Error: Cannot create an instance of an abstract class.
```

```
PS C:\Users\JCool\AppDev\TS-PRACTICE>

...
[ 90, 85, 100 ]
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Sunnyside Daycare
[ 'preschooler' ]
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

14. Member visibility

```typescript
export {};

// --- Member Visibility ---

class Base {
    isPublicImplicit = 0;
    public isPublicExplicit = 1;
    protected isProtected = 2;
    private isPrivate = 3;
    #truePrivate = 4;
}

class Subclass extends Base {
    examples() {
        this.isPublicImplicit; // Ok
        this.isPublicExplicit; // Ok
        this.isProtected; // Ok

        // this.isPrivate;
        // Error: Property 'isPrivate' is private
        // and only accessible within class 'Base'.

        // this.#truePrivate;
        // Error: Property '#truePrivate' is not accessible outside
        // class 'Base' because it has a private identifier.
    }
}

new Subclass().isPublicImplicit; // Ok
new Subclass().isPublicExplicit; // Ok

// new Subclass().isProtected;
// Error: Property 'isProtected' is protected
// and only accessible within class 'Base' and its subclasses.

// new Subclass().isPrivate;
```

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Sunnyside Daycare
[ 'preschooler' ]
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Anne Sullivan
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```

15. Static field modifiers

```typescript
export {};

// --- Static Field Modifiers ---

class Question {
    protected static readonly answer: "bash";
    protected static readonly prompt =
        "What's an ogre's favorite programming language?";

    guess(getAnswer: (prompt: string) => string) {
        // Ok
        const answer = getAnswer(Question.prompt);

        // Ok
        if (answer === Question.answer) {
            console.log("You got it!");
        } else {
            console.log("Try again...")
        }
    }
}

const quiz = new Question();
quiz.guess(() => "bash");
quiz.guess(() => "javascript");

// Question.answer;
// Error: Property 'answer' is protected and only
// accessible within class 'Question' and its subclasses.
```

```
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Sunnyside Daycare
[ 'preschooler' ]
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Anne Sullivan
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
PS C:\Users\JCool\AppDev\TS-PRACTICE> npx ts-node practice.ts
Try again...
Try again...
PS C:\Users\JCool\AppDev\TS-PRACTICE>
```